

3.3 向量检索

1. 向量数据库

RAG: chunk - embedding - indexing - vector database - retrieve - answer

向量数据库是一种专门用于存储、索引、查询和检索高维向量的数据库系统。

- 高度适合处理**非结构化数据**，如图像、音频和文本
- 具备高效存储和处理高维向量数据的能力

1.1 如何选择向量数据库

常见的向量数据库: milvus、Elasticsearch、Faiss、Chroma

进行选择时可以考虑以下因素:

1. 数据规模和速度需求
2. 持久性和可靠性
3. 易用性和社区支持
4. 成本
5. 特性
6. 安全性

向量数据库	生产定位	适合场景	核心优势	主要限制	生产项目选择建议
Milvus	开源分布式向量数据库	大规模 RAG、推荐系统、图像/多模态检索、私有化部署	专为高维向量相似性搜索设计，扩展性强，适合大规模数据	部署和运维成本较高，需要配置集群、存储、监控	如果项目数据量大、需要私有化部署、需要长期扩展，优先考虑
Elasticsearch	搜索引擎 + 向量检索能力	已有 ES 搜索体系、电商搜索、日志搜索、关键词 + 语义混合搜索	全文检索成熟，适合 BM25 + 向量的混合检索	纯向量检索性能通常不如专用向量数据库，调优复杂	如果项目本来就依赖 ES，并且重视关键词搜索与语义搜索结合，可优先考虑
Faiss	向量相似度检索算法库	单机高性能检索、算法实验、离线召回、嵌入到自研系统中	性能极高，索引算法丰富，适合底层检索能力开发	不是完整数据库，缺少原生持久化、权限、分布式、服务治理	如果你要做算法验证或自研向量检索引擎，可以用；不建议直接作为完整生产向量库
Chroma	轻量级 AI 应用向量库	本地开发、小规模 RAG Demo、个人项目、快速原型	上手简单，API 友好，适合快速搭建 RAG 应用	大规模生产能力、分布式能力和企业级治理相对有限	如果是学习、Demo、PoC、小规模知识库，优先考虑；大型生产项目需谨慎
VikingDB	云原生托管向量数据库	企业级 RAG、知识库、搜索推荐、多模态检索、火山/豆包生态项目	托管化程度高，支持大规模向量存储与检索，适合稠密 + 稀疏混合检索和多模态场景	依赖火山引擎生态，私有化和跨云迁移灵活性需评估	如果项目部署在火山引擎/豆包生态，且希望减少自运维成本，可以重点考虑

简化版：

项目需求	推荐选择
大规模私有化向量检索	Milvus
已有 Elasticsearch，想增加语义搜索	Elasticsearch
做算法实验、离线召回、自研检索底座	Faiss
快速做 RAG Demo / 本地知识库原型	Chroma
使用火山引擎 / 豆包 / 企业托管知识库方案	VikingDB

1.2 向量索引指南

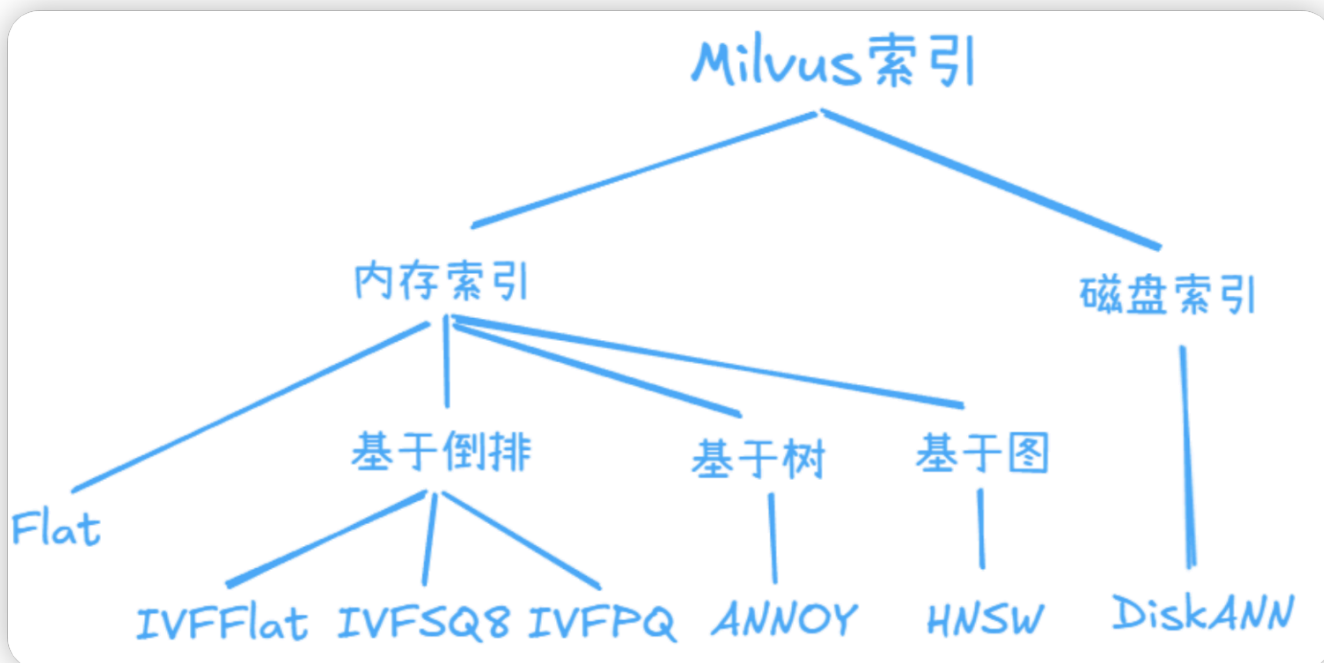
1.2.1 ANNS

最近邻搜索 (NNS) 通过给定的查询向量，快速找到数据集中最相似的若干个向量。广泛应用于：

- 图像检索
- 推荐系统
- 语音识别

但是，由于该方法会随着数据规模的增大，造成耗时增加和资源密集，近似最近邻搜索 (ANNS) 出现了。

ANNS 的核心：在可接受的精度范围内，牺牲部分准确性，换取更高的检索效率。相较于精确检索，它只需要找到目标向量的近似邻居，而不是完全精确的邻居。



1.2.2 FLAT

最简单的 Indexing 索引方法，进行暴力搜索，可以保证精确度，但效率低。

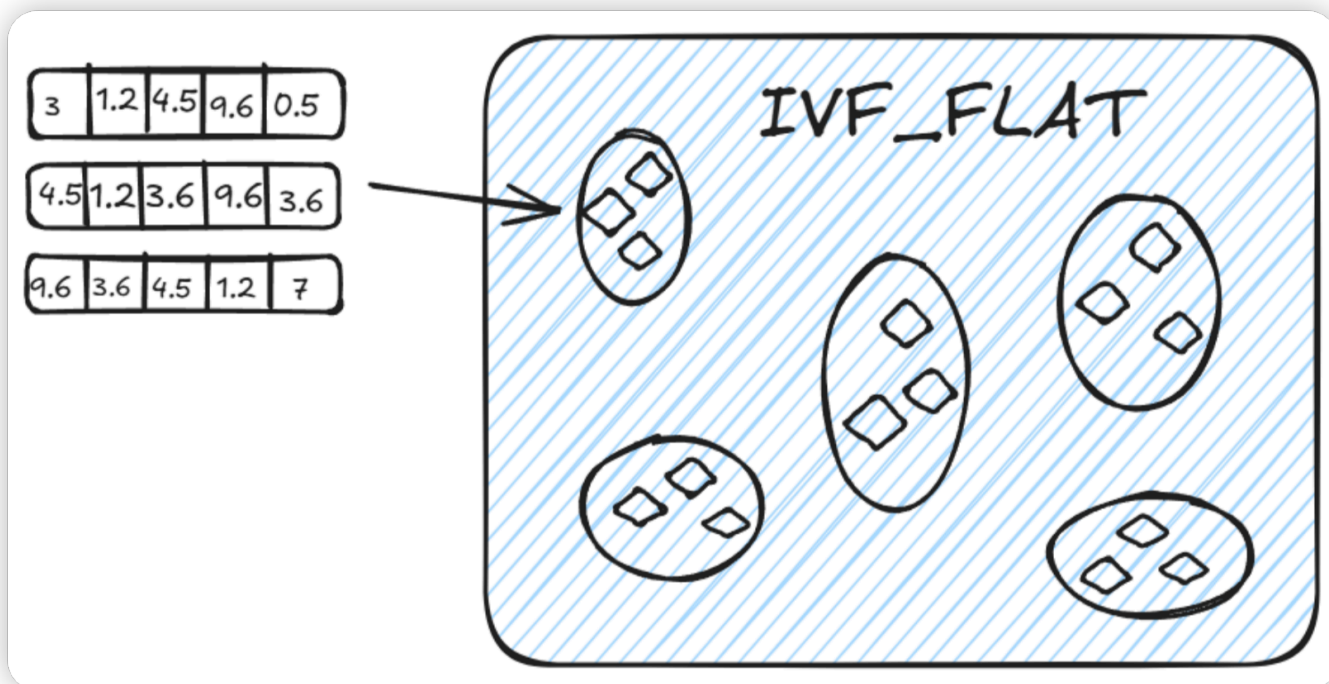
适用于: 在小型、百万级数据集上寻求完全精确的搜索结果。

1.2.3 IVF_FLAT 🌟

IVF_FLAT 基于倒排，广泛用于在大规模数据集上实现高效的 **ANNS**。

适用于: 在精度和查询速度之间寻求平衡的场景。

它本身没有进行量化操作，因此在精度和存储开销上相对保守，但能够提供较快的搜索速度。



核心原理:

1. 聚类:

- 利用聚类算法 (如 K-means) 将高维空间中的向量划分为多个子空间 (簇)。
- 每个簇包含一组相似的向量, 并且有一个代表向量, 通常是簇的中心点。

2. 倒排索引:

- 为每个簇创建倒排索引。
- 每个向量会被映射到它所属的簇, 查询时, 系统只需要关注与查询向量相似的簇, 从而显著降低搜索的时间复杂度。

3. 查询处理:

- 查询时, `IVF_FLAT` 首先将查询向量分配到距离最近的簇中心 (子空间)。
- 在该簇内部进行线性搜索, 从而查找与查询向量相似的向量。
- `nprobe` 参数用于控制搜索的簇数, 可以用于控制搜索时考虑的簇的数量, 从而平衡查询精度和查询速度:
 - 增大 可以搜索更多簇, 提高精确率, 同时提高查询时间。
 - 减少 可以缩小搜索范围, 降低计算时间, 牺牲部分精度。

4. 降低搜索成本:

- `IVF_FLAT` 通过划分子空间来限制搜索范围, 它能够显著减少传统线性搜索带来的高维数据中的计算开销。尤其适合在大规模数据集上使用。

🍰 适用于:

需要平衡精度和查询速度的场景, 尤其是在大规模、高维数据集上, 可以有效减少查询时间。

高精度, 但能容忍一定查询延迟的应用!

1.2.4 IVF_SQ8

`IVF_SQ8` 是在 `IVF_FLAT` 的基础上增加了量化步骤的缩阴方法。利用量化技术将存储和计算资源的消耗大大降低。它通过标量量化 (Scalar Quantization) 将每个维度的 4 字节浮点数表示压缩为 1 字节整数表示。

核心原理:

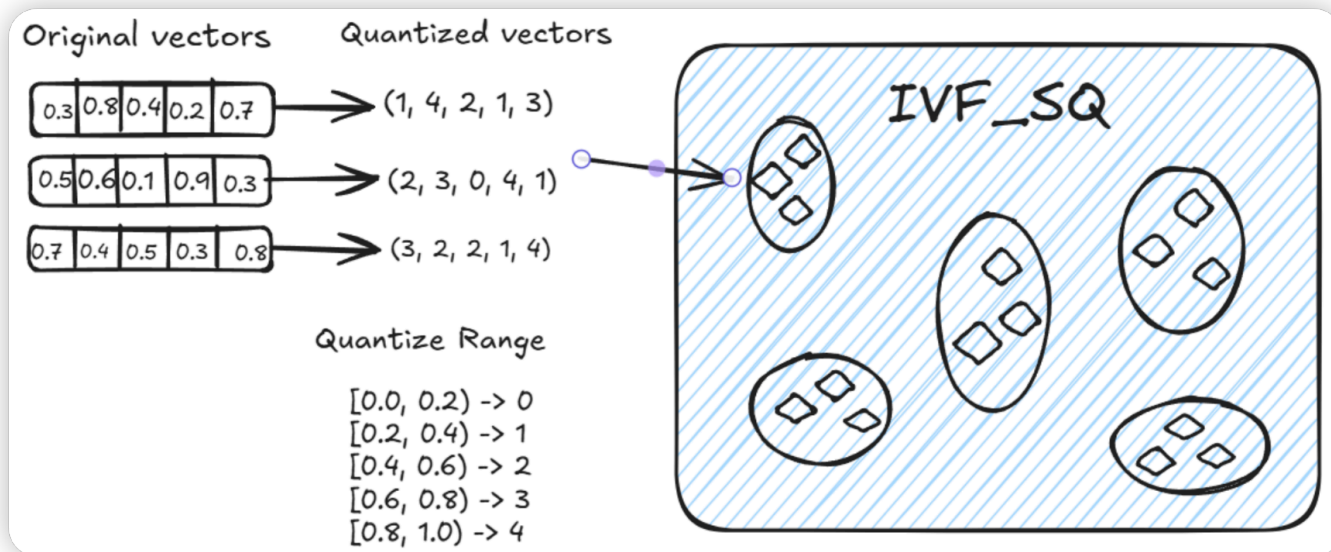
1. **标量量化**: `IVF_SQ8` 通过标量量化, 将每个向量的每个维度从 **4 字节浮点数** 压缩为 **1 字节整数**。量化过程会把原始浮点数值映射到较小的整数范围。例如, 某个维度的原始值范围是 `[0.0, 1.0]`, 量化后会被压缩为整数表示。这样可以显著节省存储空间, 并加速计算。
2. **Quantized Vectors**: 量化后的向量使用 **整数**, 例如 `uint8`, 来表示每个维度的值。通过量化, 向量的存储空间大幅减少, 同时查询时的计算量也会降低。量化后的整数会根据原始值的分布, 被划分到若干个区间中。
3. **倒排索引与聚类**: `IVF_SQ8` 与 `IVF_FLAT` 类似, 会使用 **聚类算法**, 例如 `k-means`, 将高维空间中的向量划分为多个簇。每个簇内的向量会以量化后的形式进行存储和检索。

可以理解为:

`IVF_SQ8 = IVF 倒排聚类 + SQ8 标量量化`

🌸 核心目标:

在牺牲少量精度的前提下, 减少存储空间, 并提升向量检索速度。



1.2.5 IVF_PQ

`IVF_PQ` 是一种结合了 倒排文件和乘积量化(Product Quantization, PQ), 旨在加速大规模高维数据集的检索过程。

核心原理:

1. 倒排文件

- 倒排文件可以理解为: 先给向量分组, 再去相关组里查找。
- 在 `IVF_PQ` 中, 每个向量会被分配到一个或多个倒排表中。
- 查询时, 不需要遍历全部向量, 而是先找到和查询向量最相关的倒排表, 再在其中搜索候选向量。
- 作用: 减少搜索范围, 提升检索速度。

2. 乘积量化 (PQ)

- 乘积量化可以理解为: 把高维向量拆成多个小段, 再分别压缩表示。
- 它会把一个高维向量拆成多个子空间, 并为每个子空间生成对应的量化表示, 也就是 **codebook**。
- 这样 原始向量 就不需要完整存储, 只需要存储 压缩后的编码。计算距离时, 也不直接和原始向量计算, 而是和 量化后的中心点 计算。
- 作用: 减少存储空间, 降低计算量, 加快检索速度。

3. `IVF_PQ` 的流程

1. **量化与聚类**: 每个向量被分成多个子空间, 然后对每个子空间进行乘积量化。接着, 通过倒排文件将数据按簇组织。

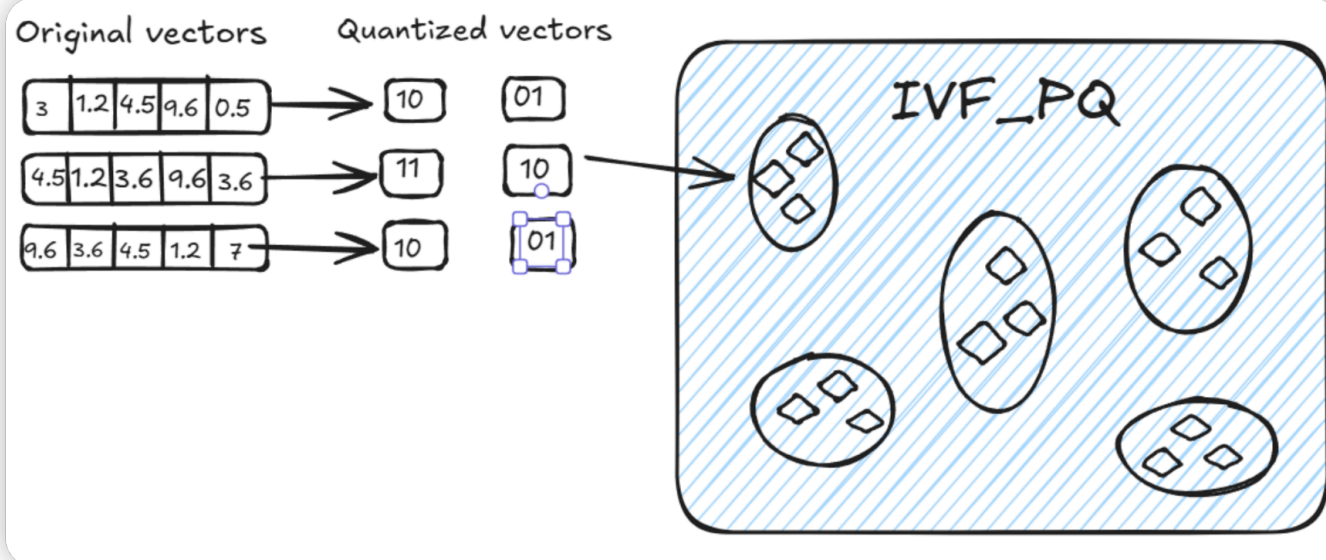
2. 查询流程:

- 先根据查询向量找到相应的倒排表 (即查询向量属于哪个簇)。
- 然后, 在该簇内, 使用乘积量化后的 代码本 来进行相似度计算, 找到与查询向量最相似的向量。

IVF_PQ = 倒排文件 IVF + 乘积量化 PQ

IVF: 负责把向量分组, 减少搜索范围。

PQ: 负责压缩向量, 减少存储和计算成本。



1.2.6 HNSW

HNSW (Hierarchical Navigable Small World Graph) 是一种基于图的索引算法, 采用分层结构和小世界图理论。它通过构建一个多层次的图结构, 其中每一层的节点连接关系不同, 逐层精细化, 从而提高高维数据集的搜索效率。

1. 图的结构

- 跳表 (Skip List)
- 可导航小世界 (NSW)

2. 跳表的特点:

- 多层链表: 底层是一个完整且有序的链表 (存储所有元素)。上层链表是下层链表的“抽象版”, 包含部分元素, 逐层变得更稀疏。
- 逐层查找: 查询时, 从最上层开始查找, 如果当前层无法找到目标元素, 则跳到下一层继续查找, 直到最底层。



3. 可导航小世界

- 邻接列表:

- 每个节点连接若干相似节点，称为邻接节点。每个节点都保存一个邻接列表。

- 遍历过程:

- 从随机选定的入口节点开始，通过图的边，逐渐找到最接近查询向量的节点。

4. HNSW 的工作原理

- 索引构建

- 图的层次结构: HNSW 会构建一个多层图。上层节点少，负责粗粒度搜索；底层节点多，负责更精细搜索。查询时会从上到下逐层接近目标向量。
- 连接邻居: 每个新加入的节点，会选择若干个近邻节点进行连接。这样可以形成局部邻接关系，保证图结构稀疏，同时提升搜索效率。

- 查询过程

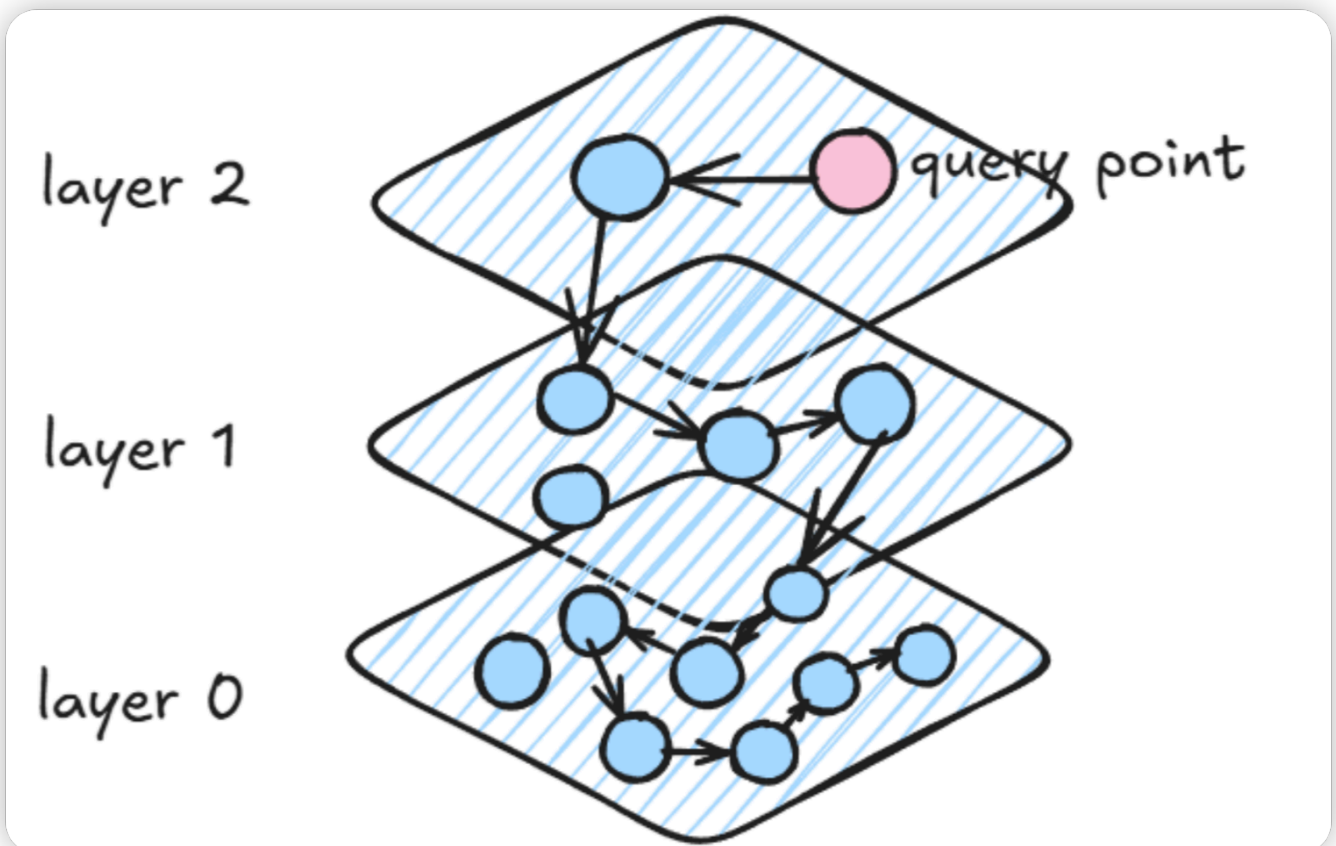
- 逐层搜索: 查询从最上层图开始，逐层向下查找。每一层都会根据相似度，从当前节点跳到更相似的邻居节点。
- 局部优化: 到达最底层后，HNSW 会在当前节点附近进行局部搜索。通过遍历邻居节点，找到最接近查询向量的结果。

HNSW = 多层图搜索

上层: 快速缩小搜索范围

底层: 精细查找最近邻

🌸 核心思想: 像“导航”一样，从粗到细逐步接近目标向量。



DiskANN 是一种 基于磁盘的高性能向量近邻搜索算法，旨在解决大规模向量数据检索中的内存消耗问题。将轻量级的索引结构置于内存中，而将海量的原始数据和构建好的图结构存放在此盘上，DiskANN能够在保持高召回率和低时延的同时，大幅减少对内存资源的依赖。

DiskANN 的优势:

- 与基于内存的算法相比:
 - 如 HNSW 和 IVF，它在资源消耗和可扩展性上有明显优势，能够在更低的资源消耗下提供相似的查询性能。
- 与基于聚类压缩的算法相比:
 - 如 IVF_PQ，它在召回率（Recall）和性能上保持高效，同时变了因压缩而导致的召回率降低。

1.3 总结

大规模、高纬度的非结构化数据检索中，要着重考虑 向量索引技术！

通过多种创新算法，有效支持了高效的 ANN 近似最近邻 搜索，尤其在大模型、推荐系统、多模态搜索等领域表现出巨大的应用潜力。

选择合适的向量检索方式依赖于具体的应用需求和数据特性，需要在性能和效率之间取得平衡，下图是一些建议：

